ECE 276B Project 3 Report

Qihao Qian

UCSD ECE Department
A69030355
q2qian@ucsd.edu

I. Introduction

This project investigates safe trajectory tracking for a differential-drive robot operating in a cluttered planar environment. The robot is required to follow a reference path of positions and orientations while maintaining a safe distance from obstacles, despite motion uncertainty caused by actuator and sensor noise. To address this challenge, we implement and compare two control strategies. The first is a receding-horizon controller that treats disturbances as nominal and solves a deterministic short-horizon optimization at each step, enabling fast response but offering limited robustness. The second models the task as a Markov decision process with discretized states and actions, where transition probabilities reflect motion uncertainty and unsafe transitions are explicitly avoided. A feedback policy is then derived through iterative policy improvement, resulting in stronger safety guarantees.

II. Problem Statement

We wish to design a feedback controller for a ground differential-drive robot so that, despite motion noise, actuator limits, and the presence of circular obstacles, the robot:

- Tracks a given reference position $r_t \in \mathbb{R}^2$ and orientation $\alpha_t \in [-\pi, \pi)$ as closely as possible;
- Always remains in the collision-free workspace $\mathcal{F}$;
- Respects its linear-velocity bound $v_t \in [0, 1]$ and angular-velocity bound $\omega_t \in [-1, 1]$;
- Minimizes a discounted infinite-horizon cost that trades off tracking error against control effort.

A. States, Controls, and Noise

True state: $x_t = \begin{bmatrix} p_t \\ \theta_t \end{bmatrix} \in \mathbb{R}^2 \times [-\pi, \pi)\,,$

Error state: $e_t = \begin{bmatrix} \tilde{p}_t \\ \tilde{\theta}_t \end{bmatrix}, \quad \tilde{p}_t = p_t - r_t,\ \tilde{\theta}_t = \theta_t - \alpha_t\,,$

Control input: $u_t = \begin{bmatrix} v_t \\ \omega_t \end{bmatrix} \in \mathcal{U} := [0, 1] \times [-1, 1]\,,$

Process noise: $w_t \sim \mathcal{N}\big(0, \mathrm{diag}(\sigma)^2\big),$

B. Error-State Dynamics

The discrete-time update (with sampling interval $\Delta > 0$) is

$$e_{t+1} = g\big(t, e_t, u_t, w_t\big) =$$

$$e_t + \underbrace{\begin{bmatrix} \Delta\ \mathrm{sinc}\big(\frac{\omega_t \Delta}{2}\big) \cos\big(\tilde{\theta}_t + \frac{\omega_t \Delta}{2}\big) & 0 \\ \Delta\ \mathrm{sinc}\big(\frac{\omega_t \Delta}{2}\big) \sin\big(\tilde{\theta}_t + \frac{\omega_t \Delta}{2}\big) & 0 \\ 0 & \Delta \end{bmatrix}}_{F(\tilde{\theta}_t, \omega_t)} \begin{bmatrix} v_t \\ \omega_t \end{bmatrix} + w_t\,.$$

C. Constraints

$$\tilde{p}_t + r_t \ \in\ \mathcal{F}, \qquad u_t \in \mathcal{U}.$$

D. Stage Cost

At each time t, define

$$\ell(e_t, u_t) = \underbrace{\tilde{p}_t^\top Q\, \tilde{p}_t}_{\text{position error}} \ + \underbrace{q\big(1 - \cos \tilde{\theta}_t\big)}_{\text{orientation error}} \ + \underbrace{u_t^\top R\, u_t}_{\text{control effort}}\,,$$

with $Q \succ 0$, $R \succ 0$, and $q > 0$.

E. Optimal Control Objective

For initial time τ and error $e_\tau = e$, compute the policy $\pi^*(t, e)$ that solves

$$V^*(\tau, e) = \min_{\pi} \mathbb{E}\left[\sum_{t=\tau}^{\infty} \gamma^{\,t-\tau}\, \ell\big(e_t,\, u_t\big) \ \middle|\ e_\tau = e\right] \tag{1}$$

subject to

$$\begin{aligned} &e_{t+1} = g(t, e_t, u_t, w_t), \\ &w_t \sim \mathcal{N}(0, \mathrm{diag}(\sigma)^2), \\ &u_t = \pi(t, e_t) \in \mathcal{U}, \\ &\tilde{p}_t + r_t \in \mathcal{F}. \end{aligned}$$

III. Technical Approach

We adopt two distinct approaches to address the problem: the Certainty-Equivalent Control (CEC) method operates in a continuous space, whereas the Generalized Policy Iteration (GPI) method is formulated in a discrete space.

A. Certainty-Equivalent Control

Optimisation variables:

$$\mathbf{U} = \begin{bmatrix} v_\tau & \omega_\tau & \dots & v_{\tau+T-1} & \omega_{\tau+T-1} \end{bmatrix}^\mathsf{T} \in \mathbb{R}^{2T},$$
$$u_k = \begin{bmatrix} v_k \\ \omega_k \end{bmatrix},\ k = 0, \dots, T-1.$$

Objective:

$$\min_{\mathbf{U}}\ J(\mathbf{U}) = \sum_{k=0}^{T-1} \gamma^k \big(\tilde{p}_k^\mathsf{T} Q \tilde{p}_k + q\,(1-\cos\tilde{\theta}_k)^2 + u_k^\mathsf{T} R u_k\big) + \gamma^T \big(\tilde{p}_T^\mathsf{T} Q \tilde{p}_T + q\,(1-\cos\tilde{\theta}_T)^2\big).$$

Deterministic error dynamics: $(k = 0, \dots, T-1)$

$$\tilde{p}_{k+1} = \tilde{p}_k + \Delta\ \mathrm{sinc}\Big(\tfrac{\omega_k \Delta}{2}\Big) \begin{bmatrix} \cos\big(\tilde{\theta}_k + \frac{\omega_k\Delta}{2}\big) \\ \sin\big(\tilde{\theta}_k + \frac{\omega_k\Delta}{2}\big) \end{bmatrix} v_k + (r_k - r_{k+1})_{1:2},$$
$$\tilde{\theta}_{k+1} = \tilde{\theta}_k + \omega_k \Delta + (r_k - r_{k+1})_3$$

Constraints:

$$v_{\min} \le v_k \le v_{\max}, \quad \omega_{\min} \le \omega_k \le \omega_{\max},$$
$$\|p_k - c_j\|_2^2 \ \ge\ (r_j + r_{\text{robot}})^2,$$
$$-3 + r_{\text{robot}} \le p_{k,x} \le 3 - r_{\text{robot}},$$
$$-3 + r_{\text{robot}} \le p_{k,y} \le 3 - r_{\text{robot}}$$

Complete NLP (receding-horizon CEC):

$$\boxed{\begin{array}{l} \min_{\mathbf{U}} J(\mathbf{U}) \\ \text{s.t. dynamics,} \\ \quad \text{input limits,} \\ \quad \text{obstacle avoidance,} \\ \quad \text{workspace bounds.} \end{array}}$$

At each time τ, this nonlinear program is solved (e.g. with SLSQP); the first control pair (v_τ, ω_τ) from the optimal $\mathbf{U}^\star$ is applied, and the horizon then recedes.If there is no best solution, the model will plan a safe control without break constrain, with big w and small v, to help robot find a way out.

B. Generalized Policy Iteration

Problem set-up: A differential–drive robot with state $x_t = (p_t, \theta_t) \in \mathbb{R}^2 \times (-\pi, \pi]$ is required to follow a reference trajectory $r_t = (r_t^p, \alpha_t)$ while avoiding circular obstacles. The discrete-time kinodynamic model with sampling period $\Delta > 0$ is

$$x_{t+1} == x_t + \begin{bmatrix} \Delta\ \mathrm{sinc}(\frac{\omega_t\Delta}{2}) \begin{bmatrix} \cos(\theta_t + \frac{\omega_t\Delta}{2}) \\ \sin(\theta_t + \frac{\omega_t\Delta}{2}) \end{bmatrix} & 0 \\ 0 & \Delta \end{bmatrix} u_t + w_t,$$

where $u_t = (v_t, \omega_t) \in \mathcal{U} = [0.1, 1] \times [-1, 1]$ and $w_t \sim \mathcal{N}(0, \mathrm{diag}\,\sigma^2)$, $\sigma = (0.04, 0.04, 0.004)^\top$.

Error dynamics: Let the tracking error be $e_t = (\tilde{p}_t, \tilde{\theta}_t)$ with $\tilde{p}_t = p_t - r_t^p$ and $\tilde{\theta}_t = \theta_t - \alpha_t$. Using equation below to obtains the error update

$$e_{t+1} == e_t + \underbrace{\begin{bmatrix} \Delta\ \mathrm{sinc}(\frac{\omega_t\Delta}{2}) \begin{bmatrix} \cos(\tilde{\theta}_t + \alpha_t + \frac{\omega_t\Delta}{2}) \\ \sin(\tilde{\theta}_t + \alpha_t + \frac{\omega_t\Delta}{2}) \end{bmatrix} \\ 0 \end{bmatrix}}_{:= \mu(t, e_t, u_t)} u_t + \big(r_t - r_{t+1}\big) + w_t.$$

Optimal control objective: For discount factor $\gamma = 0.95$ the infinite-horizon stochastic problem is

$$V^\star(\tau, e_\tau) = \min_\pi \mathbb{E}_\pi\Big[\sum_{t=\tau}^{\infty} \gamma^{t-\tau} \big(\tilde{p}_t^\top Q \tilde{p}_t + q\big(1 - \cos\tilde{\theta}_t\big)^2 + u_t^\top R u_t\big)\Big]$$

Adaptive state discretisation: The continuous error space $\tilde{p} \in [-6, 6]^2$, $\tilde{\theta} \in (-\pi, \pi]$ is discretised with *non-uniform, three-zone grids*:

$$\Delta_{\text{fine}} = 0.05 \quad \text{for } |e_x|, |e_y| \le 2,$$
$$\Delta_{\text{coarse}} = 0.15 \quad \text{for } 2 < |e_x|, |e_y| \le 6,$$
$$\delta_\theta^{\text{fine}} = 0.05 \quad \text{for } |\tilde{\theta}| \le 0.5, \quad \delta_\theta^{\text{coarse}} = 0.20 \quad \text{otherwise.}$$

Consequently the grid is dense where accurate tracking matters and sparse farther away, reducing the total number of error states to manageable size even though $T = 100$ time indices are retained.

Control discretisation: The same angular/linear resolution is applied to u, yielding N_v linear-speed points and N_ω yaw-rate points.

Transition–probability matrix. For every grid triple (t, e, u) let $\mu(t, e, u)$ be the noise–free successor obtained from **(??)** with $w_t = 0$. Denote the noise standard deviations by $\sigma = (\sigma_x, \sigma_y, \sigma_\theta)$ and set

$$d_j^2 \ = \sum_{i \in \{x, y, \theta\}} \frac{\big(e'_{j,i} - \mu_i\big)^2}{\sigma_i^2}, \qquad e'_j \in \mathcal{G},$$

where $\mathcal{G}$ is the error grid. Keep the $k = 4$ grid points $\mathcal{N}_k(\mu) = \{e'_1, \dots, e'_k\}$ with the smallest d_j^2, and assign

$$P\big(e' \,|\, t, e, u\big) = \begin{cases} \dfrac{\exp\big(-\frac{1}{2} d_j^2\big)}{\sum_{l=1}^{k} \exp\big(-\frac{1}{2} d_l^2\big)}, & e' = e'_j \in \mathcal{N}_k(\mu), \\ 0, & e' \notin \mathcal{N}_k(\mu). \end{cases}$$

The resulting sparse rows (indices e'_j, probabilities P_j) are stored in CSR format and reused throughout the GPI loop.

Stage Cost Matrix construction: A 6-D array $C(t, i_x, i_y, i_\theta, i_v, i_\omega)$ stores the expected one–step cost for every grid triple (t, e, u).

- Instantaneous loss. For a successor error $e' = (\tilde{p}', \tilde{\theta}')$ we define

$$\ell(e', u) = \tilde{p}'^{\top} Q\, \tilde{p}' + q\big(1 - \cos\tilde{\theta}'\big)^2 + u^{\top} R u.$$

- Expectation using sparse transitions. With the k nearest successors $\{(p_j, e'_j)\}_{j=1}^{k}$ from the CSR transition tables,

$$C(t, e, u) = \sum_{j=1}^{k} p_j \Big[\ell(e'_j, u) + \lambda_{\text{col}}\, \mathbb{1}_{\{\tilde{p}'_j + r^p_{t+1} \notin \mathcal{F}\}}\Big], \qquad \lambda_{\text{col}} = 10^6.$$

- Terminal step. At $t = T - 1$ only the control-effort term $(u^{\top} R u)$ is used. The full tensor is pre-computed once, saved to disk, and reused in every GPI evaluation–improvement sweep.

Collision handling:

- If either e or any sampled e' violates the safety constraint $\tilde{p} + r^p_t \notin \mathcal{F}$, the branch is discarded and the corresponding control entry marked "invalid".
- A large penalty $\lambda_{\text{col}} = 10^6$ is added to the stage cost when any successor collides, i.e. $\ell(e, u) \leftarrow \ell(e, u) + \lambda_{\text{col}}\, P_{\text{col}}(e, u)$.

Sparse data structures and parallelism:

- Each state–action row contains at most $k = 4$ non-zeros; the full transition tensor is stored as two Compressed Sparse Row (CSR) matrices: one for indices, one for probabilities.
- Pre-computation of both transitions and stage costs is carried out in joblib.parallel across all (t, e) with a job pool, after which the results are cached to disk for subsequent runs.

GPI loop:

1) **Initialisation**: value function $V^{(0)}$ set to 0; policy $\pi^{(0)}$ chosen randomly from control space.
2) **Policy evaluation**: solve the linear system

$$V^{(k)} = \mathcal{T}^{\pi^{(k)}} V^{(k)}$$

via Gauss–Seidel on the sparse rows, where

$$V_{\text{new}} = \ell^{\pi} + \gamma P^{\pi} V_{\text{old}}.$$

3) **Policy improvement**: update

$$\pi^{(k+1)}(t, e) = \arg\min_{u \in \mathcal{U}_{\text{valid}}} \Big[\ell(t, e, u) + \gamma \sum_{e'} P(e' | t, e, u)\, V^{(k)}(t+1, e')\Big].$$

4) Iterate until

$$\pi^{\text{new}} = \pi^{\text{old}}$$

Because every operator only touches the k-sparse rows, each sweep is $\mathcal{O}(k\,|S|)$ and scales linearly with the number of grid points, allowing convergence within minutes on a modern multi-core workstation.

Outcome. The resulting stationary policy $\pi^{\star}$ minimises overall cost with the cached transition and cost matrix. And could achieving high-precision tracking near the reference path while staying provably outside all obstacles

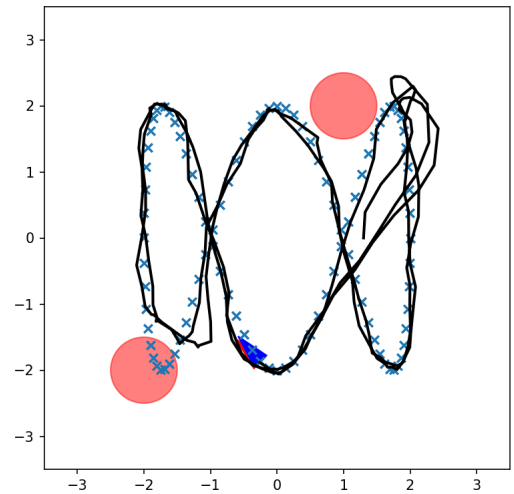

Fig. 1: Result of CEC Planner

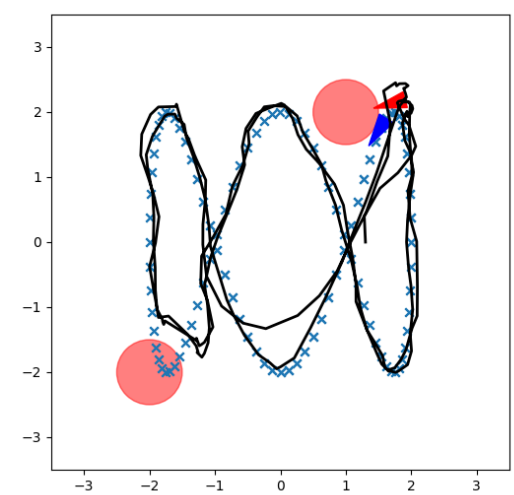

Fig. 2: Result of CEC Planner after increase position cost weight

IV. Result

A. Certainty-Equivalent Control

The code was implemented using NumPy, as CasADi led to infeasibility issues that were resolved by switching to NumPy. Furthermore, assigning excessive weight to the position cost can cause the robot to get stuck near obstacles, since it attempts to closely follow the reference trajectory but is constrained from entering the obstacle area. The result shown in Fig. 1 was generated under noisy conditions.

- Q: [1.0, 1.0]; q:1
- Total time: 16.401379108428955 s
- Average iteration time: 68.14454595247904 ms
- Final error trains: 109.77105586264355
- Final error rot: 79.45123475734296

While increase position cost weight, decrease angle loss weight, error increases.The result is show in fig 2

- Q: [2.0, 2.0]; q:0.5
- Total time: 17.617249965667725
- Average iteration time: 73.25138449668884 ms
- Final error trains: 112.12866642403705
- Final error rot: 84.69955548571875

B. Generalized Policy Iteration

I didn't define the states space well so there is a lot calculation need to do to calculate the transition matrix.But I think I have implemented a correct method.

a) Pose–error lattice.: With a time horizon of $T = 8$ steps and the non-uniform grids $S_x = S_y = 134$, $S_\theta = 49$, the pose–error space contains

$$|\mathcal{S}| = T\, S_x S_y S_\theta = 8 \times 134^2 \times 49 = 7038752 \text{ states.}$$

b) State–action rows.: Each state is combined with *two* control dimensions— the linear speed $v \in \mathcal{V}$ ($|\mathcal{V}| = 19$) and the angular rate $\omega \in \Omega$ ($|\Omega| = 41$)— so the number of rows in the transition tensor is

$$N_{\text{row}} = T\, S_x S_y S_\theta\, |\mathcal{V}|\, |\Omega| = 8 \times 134^2 \times 49 \times 19 \times 41 = 5483187808.$$

c) Sparse transition matrix.: The algorithm retains only the $k = 4$ most–probable successor states (section Technical Approach, hence

$$N_{\text{nz}} = k\, N_{\text{row}} = 4 \times 5483187808 = 21932751232.$$

So that need too much calculation

d) Practical outcome: Despite using `joblib` with all 16 physical cores (`n_jobs=-1`) and a batch size of 128, the build process (`compute_transition_matrix`) doesn't complete after $\sim 8\,\text{h}$. Consequently I have not complete the experiment.
I am so sorry.